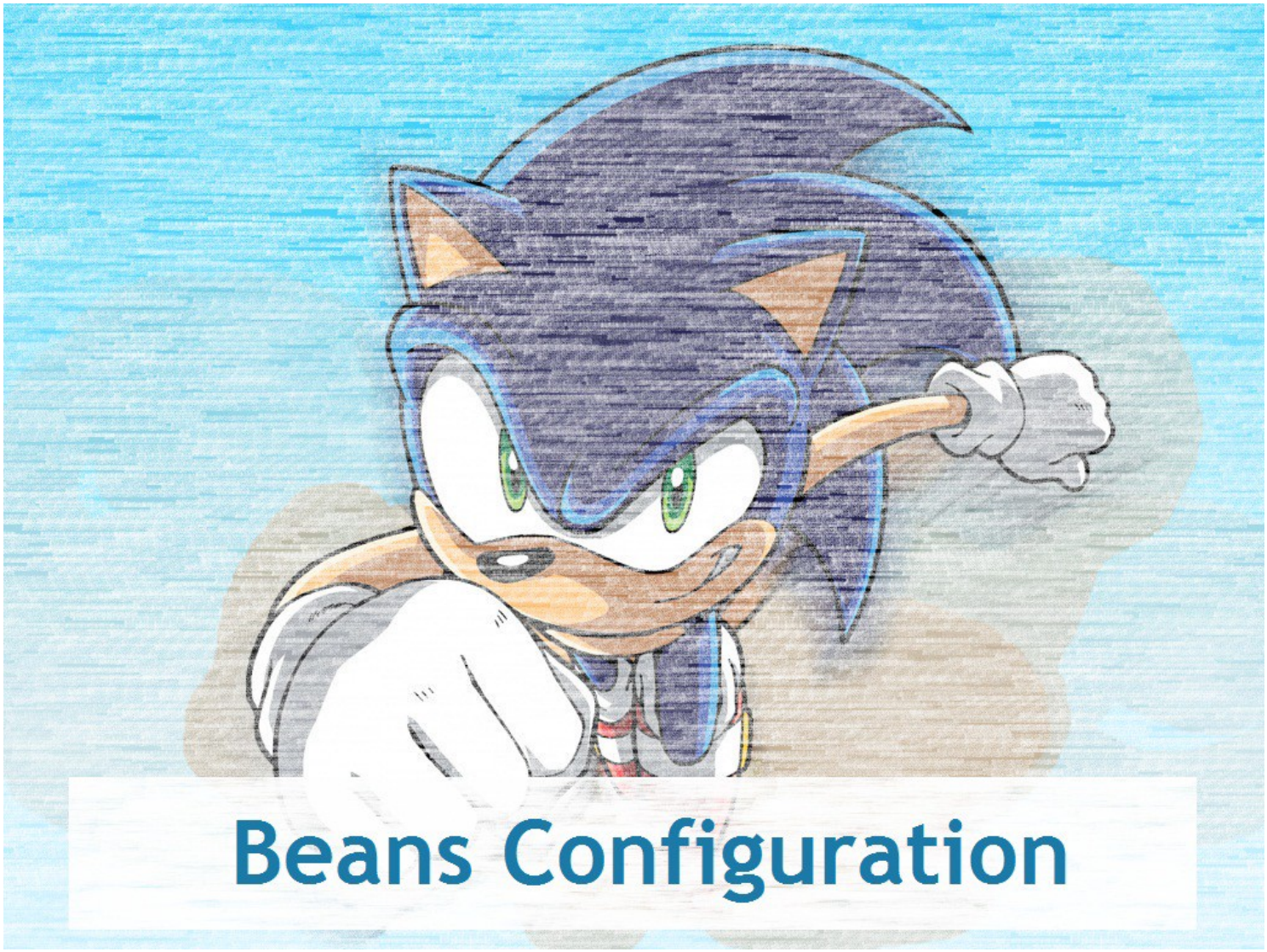
Spring: A Head Start 🔥 — Beans Configuration (Part 2)

Beans configuration using XML, Annotations, and Java code.

Omar Elgabry Follow
Sep 12, 2017 · 9 min read



Spring: A Head Start 🔥 — Beans Configuration

Spring Beans

The objects that are managed by the Spring IoC container are called **beans**. These beans are created with the configuration metadata that you supply to the container.

Bean definition contains the configuration metadata, which is needed for the container to know the following: [1] How to create a bean , [2] Bean's lifecycle, [3] Bean's dependencies.

There are 3 different ways for writing configurations: XML, Annotation, and Java Code. We'll explore each of them … 🏃

Our Project — Application Services

Below, the class diagram and code for the project we will be working on during this tutorial for demonstration on how to define the configurations.

The idea is simple, Imagine that you have an application, which relies on different services such as database, logger, mail, etc.

> 📢 *All of these files are in "src" folder of your Java application.*

```
public class App {
  private int id;
  private String name;

  public App(){
  }

 public App(int id){
  this.id = id;
 }

  public App(int id, String name){
    this.id = id;
    this.name = name;
  }

  public int getId() { return id; }
  public void setId(int id) { this.id = id; }

  public String getName() { return name; }
  public void setName(String name) { this.name = name; }
```

```
  public void setName(String name) { this.name = name; }
}
```

App.java hosted with ♡ by GitHub view raw

```
public class Database implements Service {
  @Override
  public String name() {
    return "Database MySQL";
  }

  @Override
  public void run() {
    System.out.println("Database is running");
  }

  @Override
  public void stop() {
    System.out.println("Database is stopped");
  }
}
```

Database.java hosted with ♡ by GitHub view raw

```
public class Logger implements Service {
  @Override
  public String name() {
    return "Logger X";
  }

  @Override
  public void run() {
    System.out.println("Logger is ready");
  }

  @Override
  public void stop() {
    System.out.println("Logger is stopped");
  }
}
```

Logger.java hosted with ♡ by GitHub view raw

```
public class Mail implements Service {
  @Override
  public String name() {
    return "Mail Client";
  }

```

```
  @Override
  public void run() {
    System.out.println("Mail is running");
  }

  @Override
  public void stop() {
    System.out.println("Mail is stopped");
  }
}
```

Mail.java hosted with ♡ by GitHub view raw

```
public interface Service {

 public String name();
 public void run();
 public void stop();
}
```

Service.java hosted with ♡ by GitHub view raw

```
public class Test {
 public static void main(String[] args) {
   // We will retrieve beans from Spring container here
 }
```

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:context="http://www.springframework.org/schema/context"
        xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context-3.0.xsd">

 <!-- We can define our beans here -->
</beans>
```

beans.xml hosted with ♡ by GitHub view raw

Beans Configuration — XML

[1] Define the beans

In the “beans.xml” file, we can provide definitions for beans.

```
<!-- A simple bean definition -->
<bean id="..." class="...">
   <!-- dependencies and configuration for this bean go here -->
</bean>

<!-- App bean -->
<bean id="app" class="App" lazy-init="true"></bean>
```

beans.xml hosted with ♡ by GitHub view raw

There’s a set of properties we can use for bean definition. In this example, we’ve used:

- **id:** It’s a unique identifier for the bean.
- **class**: It’s the bean class qualified name to be used to create the bean.
- **lazy-init**: It tells the IoC container to create a bean instance when it is first requested, rather than at the startup.

[2] Create Spring container

It’s used to load the beans configuration file, and create the beans.
The *ApplicationContext* is the most usually used container, while there are other containers.

— BeanFactory Container

This is the simplest container providing the basic support for DI and defined by the *BeanFactory* interface. The most commonly implementation is the *XmlBeanFactory* class (⚠ deprecated!).

The *BeanFactory* is usually preferred where the resources are limited. Otherwise, use the *ApplicationContext*.

— ApplicationContext Container

The ApplicationContext is Spring's advanced container. It includes, and extends the functionality of the *BeanFactory*. It is generally recommended over BeanFactory.

The most commonly used ApplicationContext implementations are:

- *FileSystemXmlApplicationContext*: This container loads the definitions of the beans from an XML file.
- *ClassPathXmlApplicationContext*: This container loads the definitions of the beans from an XML file (no need for the full path of the XML file).
- *WebXmlApplicationContext*: This container loads the XML file with definitions of all beans from within a web application.

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {
 public static void main(String[] args) {
    ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");
 }
}
```

Test.java hosted with ♡ by GitHub view raw

[3] Retrieve beans from Spring container

Our application will talk to the container asking to return the bean object defined in the beans configuration file using `getBean()` method.

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Test {
 public static void main(String[] args) {
    ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");
    App app = context.getBean("app", App.class);
    // App app = (App) context.getBean("app");
  }
}
```

Test.java hosted with ♡ by GitHub view raw

[4] Close the Spring container

It is important to close the spring container after use. So that all the resources will be released and all beans in its bean factory will be destroyed.

```
// How to close a Spring ApplicationContext?

context.close();
// ((ClassPathXmlApplicationContext) context).close();
// ((ConfigurableApplicationContext) context).close();
```

Spring Bean Scopes and Life Cycle — XML

Scopes defines the scope of the bean object (singleton, prototype, etc). Spring supports five scopes, three of them are available only with the web apps.

The default scope is “singleton”, while the “prototype” scope will create a new object every time we instantiate an object.

```
<bean id="app" class="App" scope="prototype"></bean>
```

> 💡*With prototype scope, beans already are lazy instantiated. So, lazy instantiation is only useful when the scope is singleton.*

Lifecycle of the bean is as the following:

Bean Instantiated → Dependencies Injected → Internal Processing → Custom Init and destroy method (hooks) → Bean is ready for use.

There are activities that take place behind the scene between the time of bean instantiation and its destruction.

When a bean is instantiated, it may be required to perform some initialization to get it into a usable state. Similarly, when the bean is no longer required and is removed from

the container, some cleanup may be required.

We'll will discuss only two important bean life cycle callback methods, which are required at the time of bean initialization and its destruction.

They are useful to make any business logic or handle resources (db, sockets, files, etc) at the beginning, as soon as object is instantiated or right before destroyed.

[1] Create the init and destroy methods

```
public class App {
 /* same as before */
 public void init(){
  System.out.println("The application has been initialized");
 }

 public void close(){
  System.out.println("Bye bye ...");
 }
}
```

[2] Define the methods in configuration

Specify the methods to be called upon instantiation and just before a bean is removed.

```
<bean id="app" class="App" init-method="init" destroy-method="close">
</bean>
```

Dependency Injection — XML

When you work on application, chances are, your object will have dependencies. Spring DI helps in wiring a class with it's dependencies, at the same time keeping them decoupled, so we can inject these dependencies during the run time.

The dependencies are defined in the bean configuration. The two most common ways to inject objects using XML: **Constructor** and **Setter Injection**.

[1] Constructor Injection

Inject dependencies to the class constructor. The IoC Container will create the object passing a dependency object to the constructor.

```
public class App {

 private Service mainService;
 private Service[] services;

  /* same as before */
  public App(Service main){
    this.mainService = main;
  }

  public App(Service[] services){
    this.services = services;
  }
}
```

App.java hosted with ♡ by GitHub view raw

In the beans.xml, define the App class dependencies.

```
<!-- App Class -->
<bean id="app" class="App">

 <!-- Constructor Initialization (one service) -->
 <constructor-arg ref="logger"/>

 <!-- Constructor Initialization (array of services) -->
 <!-- <constructor-arg>
  <list>
    <ref bean="database"/>
    <ref bean="mail"/>
    <ref bean="logger"/>
  </list>
 </constructor-arg> -->
</bean>

<!-- Service Classes (dependencies) -->
<bean id="database" class="Database"/>
<bean id="logger" class="Logger"/>
<bean id="mail" class="Mail"/>
```

beans.xml hosted with ♡ by GitHub view raw

The `constructor-arg` is used to inject dependencies and values to object's properties through the constructor. The `ref` attribute is assigned to the `id` of a bean; a reference to an object (dependency).

> ⚠ *You can't have two or more constructors in the bean definition to be used. So, In our example, we can either inject a single service, or array of services.*
>
> ⚠ Make sure to define the dependencies as beans so they can be injected.

[2] Setter Injection

Inject dependencies by calling the setter methods. The IoC Container will call the setter method of object's property passing a dependency object.

```
public class App {
  /* same as before */
  public Service getMainService() {
    return mainService;
  }
  public void setMainService(Service mainService) {
    this.mainService = mainService;
  }

  public Service[] getServices() {
    return services;
  }

  public void setServices(Service[] services) {
    this.services = services;
  }
}
```

App.java hosted with ♡ by GitHub view raw

In the beans.xml, define the class dependencies. The `property` is used to inject dependencies and values to object's properties through setter method.

```
<bean id="app" class="App">

 <!-- Setter Method Initialization -->
 <property name="mainService" ref="logger"/>
 <property name="services">
```

```
    <list>
     <ref bean="database"/>
     <ref bean="mail"/>
     <ref bean="logger"/>
    </list>
   </property>

  </bean>
```

beans.xml hosted with ♡ by GitHub view raw

📢 *How IoC knows which setter method to call? It calls the setter method with the name: The "set" keyword followed by property name with upper case first letter.*

Injecting Values — XML

We've seen how to use `constructor-arg` and `property` to inject dependencies. We can also use them to inject values.

```
<bean id="app" class="App">

 <!-- Constructor Initialization -->
 <constructor-arg type="int" value="12345"/>
 <constructor-arg type="java.lang.String" value="myApp"/>

 <!-- An easier, less-error prone way is to use the index attribute to specify the index of cons
 <!-- <constructor-arg index="0" value="12345"/> -->
 <!-- <constructor-arg index="1" value="myApp"/> -->

 <!-- Setter Method Initialization -->
 <!-- <property name="id" value="1234"/> -->
 <!-- <property name="name" value="myApp"/> -->

</bean>
```

beans.xml hosted with ♡ by GitHub view raw

The `value` attribute is assigned to the value of object's property. In case of a reference to another bean, use `ref` instead.

> 💡 *If you need to pass an empty string or null as a value, then do as the following:*

```
<bean id="app" class="App">
  <property name="name" value=""/>
</bean>

<bean id="app" class="App">
  <property name="name"><null/></property>
</bean>
```

Autowiring 🖊

We've been using `<constructor-arg>` and `<property>` to inject dependencies. Instead, we can autowire the dependencies, which helps in reducing the amount of configurations need to be written.

There are different options for autowiring that guide the Spring container on how to inject the dependencies. The default is no autowiring at all:

1. **byName**: Autowiring by property name. Spring container tries to match between object's property name and the beans id in the configuration.
2. **byType:** Autowiring by property datatype. Spring container tries to match between object's property type with exactly one of the beans of the same type. If more than one bean exists, a fatal exception is thrown.
3. **constructor:** It's byType, but, applies to constructor arguments. You can't have more than one constructor for initializing the dependencies.

```
<bean id="app" class="App" autowire="byType"></bean>
```

> 💡 *You need to write setter methods for dependencies to be injected when autowiring byName or byType.*

📢 *What if there are multiple implementing classes for the Service interface?*

In case of **`services`** (array of services implements Service interface): It won't work in case of byName. Otherwise, it will inject all the implementing objects.

In case of **`mainService`** (an object implements Service interface):

→ For **byType/constructor**, assign "autowire-candidate" attribute in the `<bean>` tag of all implementing classes to *false*, and one of them to *true*.

```
<bean id="app" class="App" autowire="byType"></bean>

<bean id="database" class="Database" autowire-candidate="false">
</bean>
<bean id="mail" class="Mail" autowire-candidate="false"></bean>
<bean id="logger" class="Logger" autowire-candidate="true"></bean>
```

→ For **byName**, either rename "mainService" in *App* class to one of the implementing classes (i.e. "logger"), and setter methods accordingly, or, rename the bean id of that class in the XML configuration to "mainService".

```
<bean id="app" class="App" autowire="byName"></bean>

<bean id="database" class="Database"></bean>
<bean id="mail" class="Mail"></bean>
<bean id="mainService" class="Logger"></bean>
```

Autowiring Limitations

- **Overriding possibility:** You can still specify dependencies using `<constructor-arg>` and `<property>` settings which will always override autowiring.
- **Primitive data types:** You cannot autowire so-called simple properties such as primitives, Strings, and Classes.
- **Confusing nature:** Autowiring is less exact than explicit wiring, so if possible prefer using explicit wiring.

Beans Configuration — Annotation

Instead of using XML to describe a bean configuration, you can move the bean configuration into the class itself by using annotations.

Annotations are special labels or markers on the relevant class, method, or field declaration. You probably have seen `@Override`, which in fact is an annotation that tells the compiler that this is an overridden method.

📏 Java Annotations Vs XML For Configuration

XML configurations can take a lot of work when there are a lot of beans, while annotations minimize the XML configurations.

Annotation injection is performed before XML injection. Thus, XML configurations will override that of the annotations.

[1] Enable class/bean/component scanning in beans configuration file

Annotation configuration is not turned on in the Spring container by default. So, we will need to enable it first in our Spring configuration file.

```
<context:component-scan base-package="com.demo.annotationsconfig">
</context:component-scan>
```

> ⚠ *I'm assuming you've created a package called "com.demo.annotationsconfig" where all of your Java classes live.*
>
> 💡 *On the other hand, The* `<context:annotation-config>` *tag scans and activates annotations for* ***already*** *registered beans in spring configuration XML file.*

[2] Add the @Component annotation to classes

Sprint scans the classes, whenever it finds `@Component` annotations, it automatically register the bean in the Spring container, with a bean id.

> 💡 *The* ***default bean id*** *is the name of the class name with lower case first letter, unless the class name starts with upper case letters (i.e. RESTFortuneService → RESTFortuneService)*

```
package com.demo.annotationsconfig;

import org.springframework.stereotype.Component;
import org.springframework.context.annotation.Lazy;

@Component
```

```
@Lazy
public class App {
}
```

App.java hosted with ♡ by GitHub view raw

[3] Retrieve beans from Spring container

Basically, It's same as XML configurations.

Spring Bean Scopes and Life Cycle — Annotations

The **scope** of a bean can be defined using `@Scope` annotation.

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope("prototype")
public class App {
 /* same as before */
}
```

For the **life cycle** of a bean, we can use `@PostConstruct` annotation as an initialization callback and `@PreDestroy` annotation as an destruction callback.

```
package com.demo.annotationsconfig;

import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;

@Component
public class App {

  @PostConstruct
  public void init(){
    System.out.println("The application has been initialized");
  }
```

```
  @PreDestroy
  public void close(){
    System.out.println("Bye bye ...");
  }
}
```

App.java hosted with ♡ by GitHub view raw

Dependency Injection — Annotations

Spring will scan all the classes with `@Component` annotation, register it as a bean, then it will inject the dependencies that have `@Autowired` annotation.

There are 3 ways to inject dependencies using `@Autowired` annotation: **Constructor Injection**, **Setter Injection**, and **Field Injection**.

> ⚠ Again! *Make sure to create beans for the dependencies, either using `@Component` annotation, or in XML configuration. Otherwise, It can't be injected.*

```
package com.demo.annotationsconfig;

import org.springframework.stereotype.Component;

@Component
public class Database implements Service {

  @Override
  public String name() {
    return "Database MySQL";
  }

  @Override
  public void run() {
    System.out.println("Database is running");
  }

  @Override
  public void stop() {
    System.out.println("Database is stopped");
  }
}
```

Database.java hosted with ♡ by GitHub view raw

```java
package com.demo.annotationsconfig;

import org.springframework.stereotype.Component;

@Component
public class Logger implements Service {
  @Override
  public String name() {
    return "Logger X";
  }

  @Override
  public void run() {
    System.out.println("Logger is ready");
  }

  @Override
  public void stop() {
    System.out.println("Logger is stopped");
  }
}
```

Logger.java hosted with ♡ by GitHub view raw

```java
package com.demo.annotationsconfig;

import org.springframework.stereotype.Component;

@Component
public class Mail implements Service {
  @Override
  public String name() {
    return "Mail Client";
  }

  @Override
  public void run() {
    System.out.println("Mail is running");
  }

  @Override
  public void stop() {
    System.out.println("Mail is stopped");
  }
}
```

[1] Constructor Injection

Inject dependencies to the class constructor using `@Autowired` annotation.

```
package com.demo.annotationsconfig;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;

@Component
public class App {

  private Service mainService;
  private Service[] services;

  public App(){
  }

  /* @Autowired
  public App(@Qualifier("logger") Service main){
    this.setMainService(main);
  } */

  @Autowired
  public App(Service[] services){
    this.setServices(services);
  }

  public Service getMainService() {
    return mainService;
  }

  public void setMainService(Service mainService) {
    this.mainService = mainService;
  }

  public Service[] getServices() {
    return services;
  }

  public void setServices(Service[] services) {
    this.services = services;
  }
```

```
}
```

> ⚠ Again! *You can't have two or more constructors to be used for autowiring.*

Now, some questions might arise …

📣 ***Question 1****: Since Service is an interface, what implementing service object (logging, etc) that will be injected?*

> *If there is only one implementing class, this will be injected by default.*
>
> *If there are multiple implementing classes, you need to use* `@Qualifier("bean_id")` *annotation to specify explicitly which object of the implementing classes to be autowired. Otherwise, Spring will choose one of the implementing classes based on property name, If no match, you'll get an error.*
>
> *In case of* **`services`** *(array of services implements Service interface): It will inject all the implementing objects.*

📣 ***Question 2****: Based on what Spring will inject the dependency?*

> *By default is byType. When using* `@Qualifier` *annotation, It's byName.*

[2] Setter Injection

Inject dependencies by calling the setter methods which has `@Autowire` annotation.

```
package com.demo.annotationsconfig;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;

@Component
public class App {

  private Service mainService;
  private Service[] services;

  public App(){
  }
```

```

  public Service getMainService() {
    return mainService;
  }

  @Autowired
  @Qualifier("logger")
  public void setMainService(Service mainService) {
    this.mainService = mainService;
  }

  public Service[] getServices() {
    return services;
  }

  @Autowired
  public void setServices(Service[] services) {
    this.services = services;
  }
}
```

You can also use `@Autowired` annotation on any method instead of the setters.

```
@Autowired
@Qualifier("logger")
public void setSomeCrazyStuff(Service mainService) {
  this.mainService = mainService;
}
```

[3] Field Injection

Inject dependencies by using `@Autowired` annotation on objects' fields.

```
package com.demo.annotationsconfig;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;

@Component
public class App {
```

```
  @Autowired
  @Qualifier("logger")
  private Service mainService;

  @Autowired
  private Service[] services;

  public App(){
  }

  public Service getMainService() {
    return mainService;
  }

  public Service[] getServices() {
    return services;
  }
}
```

App.java hosted with ♡ by GitHub view raw

Now the question is …

📢 ***Question**: Which Injection type should we use?*

> *There is no one better than the other. Just choose one of them and stay consistent in the whole project.*

Injecting Values — Annoation

We've seen how to use constructors, setter methods, and fields to inject dependencies. We can also use them to inject values.

```
package com.demo.annotationsconfig;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class App {

  @Value("125")
  private int id;

```

```
  public App(){
  }

  public App(@Value("125") int id){
    this.id = id;
  }

  public int getId() {
    return id;
  }

  @Value("125")
  public void setId(int id) {
    this.id = id;
  }
}
```

App.java hosted with ♡ by GitHub view raw

Beans Configuration — Java Code

Java code configuration enables you to write your Spring configuration without XML but with the help of Java-based annotations.

[1] Create Java configuration class using @Configuration annotation

Using the `@Configuration` tells the Spring IoC container to use it as a source of bean definitions.

> *The `@Import` annotation It allows for loading bean definitions from another configuration Java/XML file.*

```
package com.demo.javacodeconfig;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.ComponentScan;

@Configuration
// @ComponentScan("com.demo.javacodeconfig")
public class SpringConfigJavaCode {
}
```

SpringConfigJavaCode.java hosted with ♡ by GitHub view raw

> 💡 *We can add component scan using* `@ComponentScan(pacakge_name)` *annotation (optional) in the configuration file to scan all the classes within this package and create beans for all classes with* `@Component` *annotation. It works exactly same as* `<context:component-scan base-package="…">` *attribute.*
>
> 💡 *With using* `@ComponentScan` *we are going to rely on "Beans Configuration — Annotation" method, and the configuration file will be in Java class (instead of XML file).*
>
> ⚠ *So, make sure there is no conflict; no beans defined with "Beans Configuration — Annotation" method and also with "Java Code".*

[2] Define methods for creating the beans using `@Bean` annotation

The `@Bean` annotation tells Spring that this method will return an object that should be registered as a bean in the Spring application context.

The method name will be used as the bean id when Spring register this bean.

```
package com.demo.javacodeconfig;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class SpringConfigJavaCode {

 @Bean(initMethod = "init", destroyMethod = "close" )
 @Scope("prototype")
 public App app() {
  return new App();
 }
}
```

SpringConfigJavaCode.java hosted with ♡ by GitHub view raw

[3] Inject bean dependencies

We can inject the dependencies by having one bean method calling another.

```
package com.demo.javacodeconfig;
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;

@Configuration
public class SpringConfigJavaCode {

  @Bean
  public App app() {
    return new App(logger());
  }

  /* @Bean
  public App app() {
    App app = new App();

    Service[] services = {database(), logger(), mail()};
    app.setServices(services);
    app.setMainService(services[1]);
    app.setId(1234);
    return app;
  } */

  @Bean
  public Database database() {
   return new Database();
  }

  @Bean
  public Logger logger() {
    return new Logger();
  }

  @Bean
  public Mail mail() {
    return new Mail();
  }
}
```

[3] Read the configuration class

This can be done using *AnnotationConfigApplicationContext* instead.

```
package com.demo.javacodeconfig;
import org.springframework.context.ApplicationContext;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Test {
 public static void main(String[] args) {
  ApplicationContext context = new AnnotationConfigApplicationContext(SpringConfigJavaCode.class)
 }
}
```

Test.java hosted with ♡ by GitHub view raw

[4] Retrieve the bean from the Spring container

```
App app = context.getBean("app", App.class);
```

Thank you for reading! If you enjoyed it, please clap 👏 for it.

Java Spring Framework Programming Software Development Other

About Help Legal

Get the Medium app

Download on the App Store GET IT ON Google Play